Towards Refining Developer Questions using LLM-Based Named Entity Recognition for Developer Chatroom Conversations

Pouya Fathollahzadeh, *Student Member, IEEE,* Mariam El Mezouar, Hao Li, Ying Zou, and Ahmed E. Hassan, *Fellow, IEEE*

***Abstract*—In software engineering chatrooms, communication is often hindered by imprecise questions that cannot be answered. Recognizing key entities (e.g., programming languages and libraries) and user intent (e.g., learning or requesting a review) can be essential for improving question clarity and facilitating better exchange. However, existing research using natural language processing techniques often overlooks these software-specific nuances. In this paper, we introduce SoftwarE-specific Named entity recognition, Intent detection, and Resolution classification (SENIR), a labelling approach that leverages a Large Language Model to annotate entities, intents, and resolution status in developer chatroom conversations. To offer quantitative guidance for improving question clarity and resolvability, we build a resolution prediction model that leverages SENIR's entity and intent labels along with additional predictive features. We evaluate SENIR on the DISCO dataset using a subset of annotated chatroom dialogues. SENIR achieves an 86% F-score for entity recognition, a 71% F-score for intent detection, and an 89% F-score for resolution status classification. Furthermore, our resolution prediction model, tested with various sampling strategies (random undersampling and oversampling with SMOTE) and evaluation methods (5-fold cross-validation, 10-fold cross-validation, and bootstrapping), demonstrates AUC values ranging from 0.7 to 0.8. Key factors influencing resolution include positive sentiment and entities such as `Programming Language` and `User Variable` across multiple intents, while diagnostic entities (e.g., `Error Name`) are more relevant in error-related questions. Moreover, resolution rates vary significantly by intent: questions about *API Usage* and *API Change* achieve higher resolution rates, whereas *Discrepancy* and *Review* have lower resolution rates. A Chi-Square analysis confirms the statistical significance of these differences.**

***Index Terms*—Empirical Software Engineering, Developer Chatroom, Named Entity Recognition, Large Language Models, Question Resolution, Mixtral**

I. Introduction

Developer chatrooms, such as Discord[1] and Slack,[2] are crucial tools for collaboration and knowledge sharing in software development. These platforms enable developers to seek help, solve technical problems, and engage continuously with the community. Collective knowledge available in chatrooms is shown to accelerate software development and improve project outcomes [1], [2]. Additionally, chatrooms help build and maintain active developer communities, which are vital to the long-term success of open-source projects [3].

Despite the important role of chatrooms, their effectiveness is often hindered by communication issues. Poorly articulated questions that lack clarity or essential details frequently lead to misunderstandings, incomplete responses, or no response at all. For instance, research on Gitter[3] chatrooms finds that around 40% of questions remain unanswered [4] and delays in resolving problems discourage community participation [5]. While question-and-answer (Q&A) platforms like Stack Overflow are well studied [6], [7], [8], research on developer chatrooms is still limited. Initial findings suggest that including URLs can reduce response rates, whereas user mentions improve the rates [4]. Recent work by Lill et al. [9] has explored automated methods to improve developer support in chatrooms by identifying similar past conversations.

In developer chatrooms, a *conversation* consists of an initial question or statement (e.g., asking for help or seeking clarification) followed by responses, clarifications, and follow-ups until the question is resolved or left unresolved. The quality of the initial question impacts resolution outcomes, as well-formed questions often include sufficient technical details, such as code snippets or error types, making them easier to address. Named Entity Recognition (NER) is a useful method for extracting such technical details by identifying and categorizing domain-specific entities, such as libraries and error messages [10], [11]. While traditional NER methods work well on structured and formal text, they often require extensive feature engineering and task-specific training datasets, limiting their adaptability to informal and fragmented chatroom conversations [12], [13], [14], [10]. In contrast, Large Language Models (LLMs) are good at understanding complex natural language and generalizing across diverse contexts due to their pretraining on vast corpora [15], [16], [17].

To address these challenges, we propose an approach named SoftwarE-specific Named entity recognition, Intent detection, and Resolution classification (SENIR) that leverages an LLM to label the chatroom conversations. SENIR recognizes software-specific entities, detects the intent behind

Pouya Fathollahzadeh and Ying Zou are with the Department of Electrical and Computer Engineering, Queen's University, Kingston, ON K7L 3N6, Canada. E-mail: {pouya.fathollahzadeh, ying.zou}@queensu.ca.

Hao Li and Ahmed E. Hassan are with the Software Analysis and Intelligence Lab (SAIL), School of Computing, Queen's University, Kingston, ON K7L 3N6, Canada. E-mail: hao.li@queensu.ca, ahmed@cs.queensu.ca.

Mariam El Mezouar is with the Department of Mathematics and Computer Science, Royal Military College of Canada, Kingston, ON K7K 7B4, Canada. E-mail: mariam.el-mezouar@rmc.ca.

[1]https://discord.com

[2]https://slack.com

[3]https://gitter.im

a question in the context of developer chatrooms [18], and classifies whether a conversation has reached a satisfactory conclusion (i.e., resolved) or remains unresolved. This study evaluates SENIR using the DISCO dataset [19], a collection of 29,243 developer chatroom conversations extracted from various channels related to software engineering (SE) on Discord. Our work analyzes the developer chatroom conversations along the following three research questions (RQs):

RQ1: How effective is SENIR in labelling developer chatroom conversations?

Chatroom conversations are by nature informal, fast-paced, and lack sufficient context, making automated labelling a challenging task. To address this, we design SENIR using Mixtral 8x7B [20] to perform software-specific NER, intent detection, and resolution status classification. SENIR is evaluated on a manually labelled subset of 400 Discord conversations, achieving high accuracy (91% for NER, 76% for intent detection, and 93% for resolution status), with corresponding F-scores of 86%, 71%, and 89%.

RQ2: What features of the developer questions contribute to their resolution outcomes?

We use SENIR to automatically label 29,243 developer conversations from the DISCO dataset to identify software-specific entities, intents, and resolution status. The labelled entities and intents from questions are used to engineer features together with additional features such as sentiment, posting time, and question length. These question-related features are used to train a mixed-effect model to predict resolution outcomes. The model achieves an AUC of 0.75 and the feature importance analysis reveals that positive sentiment and technical specificity (e.g., use of library functions) positively impact resolution, while late posting times and excessive URLs negatively influence resolution.

RQ3: How do entities, intents, and their interactions impact the resolution of developer questions?

We analyze the 29,243 labelled conversations to examine how entities and intents influence resolution success. Chi-Square tests reveal significant differences in resolution rates across intents, with *API Usage* and *API Change* demonstrating better resolution rates (33.6% and 26.2%) than *Discrepancy* and *Review* (22.0% and 18.8%). Within intents, specific entity pairs show better resolution outcomes than others. For instance, the pair (`Programming Language`, `Library Function`) is particularly effective for technical intents like *Errors* (63.6%), while (`UI ELEMENT`, `Website`) shows much lower success for intents like *Discrepancy* and *Review*.

Novelty & Positioning. Unlike prior work that tackles a single task (e.g., NER only, often trained on Stack Overflow [7], [14], [21]) or assumes structured Q&A [22], [23], [24], [25], [26], our study introduces the first end-to-end *zero-shot* pipeline for *informal, multi-threaded chatrooms* that *jointly* labels software-specific entities, developer intents, and conversation resolution without task-specific training data (Section III-A). We then *close the loop* from labelling to actionable refinement by (i) training a *mixed-effect* model on the *initial question only* to predict resolution while controlling for cross-chatroom heterogeneity (Section III-B), and (ii) deriving guidance from feature importance and intent-conditioned *entity-pair* analyses (Section III-C). We release the annotated dataset and pipeline to support replication and follow-on work.

TABLE I
LIST OF CHANNELS IN DISCO DATASET

Channel	Number of Conversations
python#python-general	19,684
gophers#golang	8,860
racket#general	538
clojurians#clojure	161
Total	29,243

The main contributions of our paper are as follows:

1) We propose SENIR, an LLM-based approach to automatically label developer chatroom conversations with software-specific named entities, intents, and resolution status.
2) We provide a predictive model for predicting resolution status based on the questions posted and reveal the most influential features for a question to be resolved.
3) By investigating how specific entities and user intents influence question resolution, we provide actionable insights to help developers refine their questions to potentially achieve higher response rates.
4) We release the annotated developer conversation dataset which is derived from DISCO and augmented with the SENIR labels in our replication package [27] at https://github.com/Software-Evolution-Analytics-Lab-SEAL/TSE2025-SENIR.

The rest of the paper is organized as follows: Section II outlines our methodology, including the case study design and data analysis. Section III presents the results for each research question. Section IV discusses the implications of our findings. Section V discusses related work. Section VI discusses the threats to validity. Section VII concludes the paper.

II. CASE STUDY DESIGN

This section presents our systematic approach for refining developer questions in chatrooms by leveraging an LLM. Fig. 1 presents an overview of our approach.

A. Data Collection and Preprocessing

To systematically analyze chatroom conversations, we first collect and preprocess data from the DISCO dataset [19], ensuring that our study focuses on relevant developer discussions. Table I presents an overview of the dataset.

Step 1: Extracting messages and metadata. We extract individual messages along with their associated conversation IDs, timestamps ("ts"), and user IDs from the DISCO dataset. For instance, a message like "How do I install X?" might include metadata such as "Mia" (the user who sent the message), "Aug 15, 2020, 10:34 AM" (the time it was sent), and "12345" (the conversation ID to which it belongs). Fig. 4 in Appendix A provides an example of a real conversation from the "racket#general" channel of Discord.

Step 2: Aggregating messages by conversation ID. Since each message is recorded separately without direct references

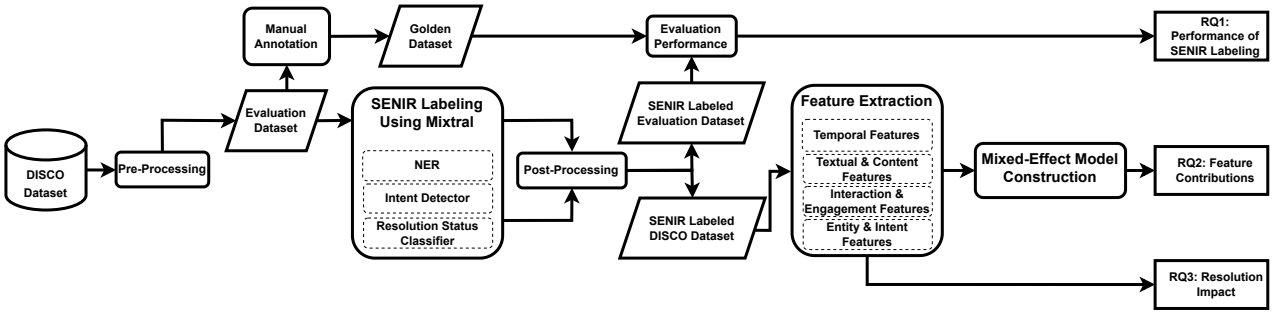

Fig. 1. Overview of our case study design.

to other messages, we group messages sharing the same "conversation ID" to reconstruct complete interactions. This aggregation enables further analysis (e.g., resolution status classification) at the conversation level rather than the individual message level.

Step 3: Augmenting conversations with additional metadata. To better understand when and over what period a conversation took place, we augment each conversation record with additional metadata. For instance, we calculate the range of months over which a conversation took place (e.g., "Aug2020–Oct2020") based on the start and end dates. This temporal information is used in two ways: (i) duration (start to end time) helps us describe how long discussions last, and (ii) the start time allows us to count how many new conversations begin in each period, which we use to study activity and frequency. We note that duration is not used as a proxy for frequency. A sample JSON can be found in Appendix A.

B. Golden Dataset Collection and Manual Labelling

We construct a golden dataset of software engineering-related chatroom conversations to evaluate the performance of the LLM in automatically labelling software-specific named entities, intent, and resolution status of a conversation. To ensure a representative sample, we randomly select a statistically representative subset of the collected conversations (as listed in Table I) with a confidence level greater than 95% and a margin of error of 5% [28], [29]. This results in a selection of 400 conversations randomly chosen from the dataset. We manually verify each conversation to ensure that it begins with a software engineering-related question.

The manual labelling process involves annotating conversations with software-related entities, intents, and resolution statuses. Two Ph.D. annotators in software engineering independently annotate the dataset to ensure a broad and unbiased perspective. Annotator A has four years of experience conducting software engineering research. Annotator B is a Ph.D. candidate with over three years of experience in software engineering research. The annotation process is conducted in two phases to ensure consistency and minimize bias:

Phase 1: Initial annotation and agreement check. The annotators first independently label a subset of 200 conversations, covering software-specific entities, intents, and resolution statuses. After that, we calculate inter-annotator agreement using Cohen's Kappa [30] to measure consistency in labels. The initial Kappa scores are 0.71 for entity recognition (substantial agreement), 0.67 for intent detection (substantial agreement), and 0.76 for resolution status (substantial agreement). Disagreements, such as different interpretations of ambiguous intents like *Conceptual*, are discussed, and labelling guidelines are refined to improve clarity and ensure alignment (e.g., specifying cues for distinguishing *Learning* from *API Usage*).

Disagreement resolution protocol. After independent double-coding, disagreements are reconciled by consensus during the three adjudication meetings described in the following paragraph. For each conflict, the annotators consult the written rules and edge-case examples. When a rule is insufficient, it is refined and the case is re-labelled accordingly. Intents are constrained to the seven pre-defined categories. During Phase 1, we track candidates that do not obviously fit in with the taxonomy. After discussion, all are mapped to the existing intents once boundary rules are added, and no new intent category is introduced. No items remain unresolved after adjudication.

Refined labelling guidelines (development and refinement). The labelling process starts from compiling an initial draft from prior work on SE-specific NER [14], [31], [32], [11] and intent categories [7], [33], [34], [35], [36], [37], defining 28 entity types and 7 intent categories with examples. We then hold three structured adjudication meetings to refine the guidelines: (i) *Resolution rules* (session 1): clarified evidence for "resolved" (direct confirmation, accepted fix) vs. "unresolved" (no actionable answer, abandonment). (ii) *Intent rules* (session 2): added decision rules and cues, e.g., *Learning* targets tutorials or foundational know-how ("Where can I find a Python tutorial?"), while *Conceptual* probes behavior/design rationale ("Why does Python's GIL work this way?"). (iii) *Entity rules* (session 3): added boundary conditions and positive/negative examples, e.g., `File Type` vs. `File Name` ("csv", "json" → `File Type`; "data.csv" → `File Name`); `Version` requires a product-tied version token ("Python 3.10", "v1.2.3"), while dates are not `Version`. Each rule was recorded with keyword cues, edge cases, and short examples to maximize consistency.

Scope of intents and auxiliary cues. Intents are constrained to the seven literature-based categories. Annotators use auxiliary signals like: code blocks/stack traces, explicit error tokens, artifact mentions (API/library/version) and speech-act cues such as "how to…", "why does…", "please review…" or migration verbs to decide among the seven intents rather than introduce new ones. During Phase 1, we flag off-taxonomy

candidates. After refining boundary rules, all technical items are mapped to existing intents.

Phase 2: Final annotation. Using the refined guidelines from Phase 1, the annotators label the remaining 200 conversations. The final Agreement improved to Cohen's Kappa of 0.84 for entity recognition (near-perfect agreement), 0.78 for intent detection (substantial agreement), and 0.87 for resolution status (near-perfect agreement). To verify representativeness, we analyze the label distribution in the golden dataset and compare it with the full DISCO dataset. The results confirm that both majority and minority labels (e.g., *Review*, *API Change*) are represented in the golden dataset. Full distribution tables are provided in Appendix A. The manual labelling process consists of the following steps:

Step 1: Identifying software-related entities in the initial question of conversations. Each annotator manually identifies and labels software-related entities within the initial questions of conversations. For each identified entity, the annotator records the exact word representing the entity (e.g., `Library`) along with the timestamp of its occurrence in the conversation. This ensures precise tracking of where and when the entity appears in the discussion.

Step 2: Categorizing intents in the initial question of conversations. Each conversation is labelled based on its initial question with one or more intents that indicate its underlying purpose. The intent annotation follows pre-defined categories (Table XI in Appendix A) to maintain consistency.

Step 3: Annotating resolution status for conversations. Unlike Stack Overflow, Discord or similar chatrooms do not have a flag to indicate whether a question is resolved. To address this, each annotator manually labels the resolution status of each conversation. A conversation is labelled as "resolved" if the original question receives a relevant answer, and as "unresolved" if it does not.

C. SENIR for Labelling Developer Chatroom Conversations

We design SENIR to label developer chatroom conversations, focusing on three tasks: **(1) Software-Specific NER:** NER identifies key software-related entities, such as `Library`, `Error Name`, and `Version`, which are crucial for understanding the context of developer discussions. **(2) Intent Detection:** Intent detection determines the purpose of each conversation, such as whether a user asks to learn a programming language or deals with an error. Understanding the intent of the initial question allows for better classification of conversations. **(3) Resolution Status Classification:** This classification task assesses whether a conversation has successfully resolved the initial question. Classifying resolved and unresolved questions helps evaluate the effectiveness of the chatroom and identifies questions requiring further attention. Throughout this paper, we follow a consistent notation for entities and intents: entities are written in `typewriter` (e.g., `Programming Language`), while intents are written in *italics* (e.g., *API Usage*).

Design rationale. SENIR is an end-to-end, *zero-shot* prompting pipeline that operates on unstructured chatroom conversations and *jointly* produces three labels: SE-specific entities, intents, and resolution within a single framework. This joint formulation is tailored to Discord-like chats (informal, fragmented, multi-topic), where isolated NER- or intent-only models trained on structured Q&A struggle to generalize.

Step 1: Selecting the LLM. Although SENIR is compatible with various LLMs, we initially selected Mixtral 8x7B [20] due to its large context window (33k tokens) which allows us to process full multi-turn Discord conversations without truncation compared to other open-source models on the chatbot arena leaderboard [38] as of January 23rd, 2024, when we started our research. The context window size refers to the maximum number of tokens that a model can process at one time when generating a response. A larger context window allows the model to take into account longer sequences of text, which is important when processing developer conversations that may contain up to 100 messages.

To further examine this choice, we also evaluated two other long-context open-source models (ChatGLM3-6B-32k and LongChat-v1.5-7B-32k) on our golden dataset for both NER and intent detection. The empirical backbone comparison is reported in Appendix B.

Step 2: Constructing prompts. To optimize performance, we design custom prompts to guide the LLM in handling specific tasks. The prompts include clear rules that instruct the model on what to focus on. For instance, in the NER prompt, the rules guide the LLM on which entities to extract (e.g., `Programming Language`). By following best practices in prompt engineering [39], [40], we enhance the model's ability to generate accurate and contextually relevant outputs.

To ensure consistent and accurate outputs, we design our prompts using a structured, instruction-style format with a strict output schema and a closed ontology. First, each prompt begins with a clear task specification that defines the objective, such as extracting entities, and specifies the required output. We explicitly instruct the model to return JSON output only, without any accompanying rationale or explanation. Second, we use closed label sets by listing all 28 entity types and 7 intent categories verbatim within the prompt, requiring the model to select from these exact strings rather than generating free-form labels. Third, we enforce a strict JSON schema with fixed field names and formats to ensure that the output is structured and easily parsable. For example, `{"entities": ["text": "...", "type": "..."]}` for NER and `{"intents": ["..."]}` for intent detection. Finally, for tasks related to RQ1, we use only the initial question as input to keep prompts compact and minimize off-topic drift. The prompts used in this paper are available in our replication package [27].

NER prompt example: The model receives the initial question of a conversation along with timestamps, and is tasked with recognizing named entities. We collect a list of 28 software-specific entities based on prior work [14], [31], [32], [11] and include this pre-defined list to guide entity recognition. Table II provides examples and references for each of these entities.

TABLE II
SOFTWARE-SPECIFIC ENTITY CATEGORIES.

Entity	Examples	Reference
`Application`	Mosh, JKplayer, api-java-client	[14], [31], [32], [11]
`Programming Language`	Python, Java, CSS, C++	[14], [31], [32], [11]
`Version`	(Python) 2.7, (Windows) XP	[14], [31]
`Algorithm`	UDP, DFS, RBM	[14]
`Operation System`	Linux, iOS, Windows	[14], [31], [32], [11]
`Device`	Phone, Mobile, GPU	[14], [31], [32]
`Error Name`	Overflow, OutofRange, I/O Error	[14]
`User Name`	John, Maya, Clark, @Maya	[14], [32]
`Data Structure`	Array, List, Hash table, Heap	[14]
`Data Type`	String, Char, Double	[14]
`Library`	Numpy, Scipy, Auto-grad	[14], [32], [11]
`Library Class`	ItemTemplate, actionManager	[14], [32], [11]
`User Class`	myClass, TestClass	[14], [31]
`Library Variable`	math.inf, swing.color, ActionListener.Value	[14]
`User Variable`	user id, numberOfRowsInSection	[14]
`Library Function`	numpy.isinf(), Math.floor()	[14], [32], [11]
`User Function Name`	hello(), myFunction()	[14], [32], [11]
`File Type`	JSON, JAR	[14], [32], [11]
`File Name`	WindowsStoreProxy.xml, a.txt	[14]
`UI Element`	Button, Scroll bar, Text box	[14]
`Website`	MSDN, Google, Yahoo	[14]
`Organization`	Apache, Microsoft Research, Fair	[14], [31]
`License`	CC BY 4.0, Apache 2.0	[14], [11]
`HTML/XML Tag Name`	h1, div, img	[14], [32], [11]
`Value`	"hello world", 255, 30.5, True	[14]
`In Line Code`	grep -rnw, select * from Tab	[14]
`Output Block`	Output from console/any IDE	[14]
`Keyboard Input`	CTRL+X, ALT, fn	[14]

Prompt: Extract all relevant software entities *[list of entities]* from the text below.
Input Format:
Question: I am using TensorFlow version 2.5 and encountering an error during installation. How can I fix this?
Output Format:
["TensorFlow: Library", "2.5: Version"]

Intent detection prompt example: The model receives the initial question of a conversation and is tasked with detecting the intent. A list of 7 categories of pre-defined intents is used to label the conversation with one or more intents. The 7 categories are collected based on previous work [7], [33], [34], [35], [36], [37] and are detailed in Table XI (Appendix A).

TABLE III
COMPARISON OF PROMPTING STRATEGIES ON THE GOLDEN DATASET

Prompting Strategy	**Intent F1**	**NER F1**
Zero-shot instruction-style	71%	86%
Few-shot (3–5 examples)	68%	82%
Zero-shot chain-of-thought	65%	77%
Few-shot chain-of-thought	66%	79%

Prompt: Identify the intents behind the question based on the following categories: [list of intents].
Input Format:
Question: How to install library X?
Output Format:
["Learning"]

Resolution status classification prompt example: We also aim to use the LLM to classify whether a conversation is "resolved" or "unresolved" based on the content of the full conversation.

Prompt: Determine if the issue discussed in the following conversation is resolved.
Input Format:
- User 1: How do I fix this bug in Library Y?
- User 2: You should try reinstalling the dependency using version 3.0.
- User 1: That fixed the issue, thanks!

Output Format:
["Resolved"]

Prompting strategy comparison. Beyond the instruction-style zero-shot prompts used in our main setup, we systematically compare three alternatives: (i) few-shot prompting with 3 to 5 labeled examples, (ii) zero-shot chain-of-thought (CoT) with "think step by step" guidance, and (iii) few-shot CoT (examples + stepwise reasoning). Table III summarizes Intent and NER F-scores for each strategy. Our evaluation on the golden dataset confirm that the zero-shot approach performs best, achieving an F-score of 71% for intent detection and 86% for NER. In contrast, the other strategies perform less effectively, with few-shot CoT yielding an F-score of 66% for intent and 79% for NER. We hypothesize that for informal developer chatroom conversations, the inclusion of examples and explicit reasoning steps introduces verbosity and off-topic cues from the conversational text, which reduces precision and stability. Accordingly, we adopt the concise, instruction-style zero-shot prompts for all main experiments.

D. Feature Extraction

Feature extraction helps analyze and understand whether a conversation in the DISCO dataset can be resolved or not when the initial question is posted. The extracted features capture key aspects of the initial question, such as timing, content quality, interaction levels, and the presence of specific entities and intents. By linking these features to resolution outcomes, we gain insights into what makes a question more likely to be resolved. Feature extraction follows two steps:

Step 1: Labelling the DISCO dataset using SENIR. We run SENIR on the entire dataset (illustrated in Fig. 1) which consists of 29,243 conversations to label the entities, intents, and resolution statuses for each conversation.

Step 2: Extracting features from the labelled DISCO dataset. We engineer and extract a set of features from the labelled dataset and categorize them as temporal features, textual & content features, interaction & engagement features, and entity & intent-related features. Each category and its related features are listed in Table XII (Appendix A) and described below. Except for entity & intent-related features, all extracted features are collected from previous work [4], [22], [41], [42], [43], [44], [45], [46], [26], [47], [48].

E. Evaluation for SENIR Labelling Accuracy (RQ1)

We evaluate the labelling performance of SENIR on the golden dataset and compare labelling output from SENIR with manual labels using the performance metrics *Precision*, *Recall*, *F-score*, and *Accuracy* [49]. To calculate the evaluation metrics, we map SENIR's predictions and the golden labels into the confusion matrix categories: *True Positive* (TP), *False Positive* (FP), *False Negative* (FN), and *True Negative* (TN). The specific interpretation of TP, FP, FN, and TN varies by task (entity recognition, intent detection, and resolution status). The corresponding confusion matrix category definitions for each task are provided in Appendix B.

To understand how traditional NER and classification models handle the informal and noisy nature of Discord conversations compared to SENIR, we compare SENIR's performance against supervised baselines for both NER and intent detection. For NER, we evaluate SoftNER [14] in two configurations: (i) an off-the-shelf setting using the original model trained on Stack Overflow data, and (ii) an in-domain setting where we retrain SoftNER specifically on our 400-conversation golden dataset. For intent detection, we fine-tune a BERT classifier (bert-base-uncased) on the golden dataset to serve as the supervised baseline. We follow a standard 80-20 train-test split, resulting in 320 samples for training.

F. Resolution Prediction Modelling Setup (RQ2)

To answer RQ2, we follow the steps listed below:

Step 1: Labelling the conversations. We use the approach from RQ1 to label 29,243 conversations sourced from four distinct Discord channels within the DISCO dataset (see Table I). Each conversation is annotated with software-specific entities, the intent of the question, and the resolution outcome. The resolution outcome is used as the dependent variable in building the prediction model.

Step 2: Feature extraction. For each conversation, we isolate the initiating question and extract a comprehensive set of features from the initial questions, as described in Section II-D. These features include general attributes, such as question length and sentiment, as well as features derived from the labelling in Step 1, such as entities count in a given question. Notably, the features pertain solely to the question itself. The model is not trained on any features irrelevant to the initial question as a whole. The feature set consists of 50 question-related features, which are listed in Table XII (Appendix A).

Step 3: Feature processing. We apply max-min normalization to scale all features to a range of [0, 1]. After normalization, we perform a correlation and redundancy analysis to identify the highly correlated and redundant features.

We use Spearman's correlation coefficient to measure the degree of correlation between features because many of our extracted features (e.g., counts of URLs, entities, and code snippets) are not normally distributed and typically exhibit skewed distributions. Unlike Pearson, which assumes normality and linearity, Spearman's correlation is non-parametric and evaluates monotonic relationships, making it more robust for identifying associations in software engineering metrics. Following the previous studies [50], [51], [52], we consider a correlation coefficient threshold of 0.7 for the correlation coefficient values to identify strong correlations. The Features correlation matrix dendrogram can be found in Appendix A.

To detect redundancy, we compute the Variance Inflation Factor (VIF), a widely used metric that quantifies how much a given feature is explained by other independent variables. Following prior work [53], [54], we adopt a cut-off threshold of 10, where VIF values above this level indicate substantial redundancy. Features exceeding this threshold are excluded to improve computational efficiency and model generalizability.

After applying both correlation and redundancy filtering, we remove 15 highly correlated and redundant features: Text-Code Ratio Question, User Mentions, Total Entities Count, Unique Entities Count, Entity Occurrences, Sentiment, `License` Intent Total Count, *API Usage*, *Conceptual*, *Discrepancy*, *Errors*, *Review*, *API Change*, and *Learning*. This process reduces the feature space from 50 to 35 features while maintaining key predictive information.

Step 4: Model training. The dataset consists of a total of 29,182 questions, with 22,748 labelled as "unresolved" (77.95%) and 6,434 labelled as "resolved" (22.05%). This class imbalance is addressed through sampling strategies during model training. To account for the variability of data sourced from different chatrooms, we train a mixed-effect model that incorporates: i) fixed effects, which capture the influence of extracted features, and ii) random effects, which address variations across chatrooms. To refine the model and prevent overfitting, we employ the stepwise regression algorithm [55]. Starting with an empty model, features are iteratively added based on their contribution to predictive performance. Specifically, we employ a stepwise feature selection approach, where features are evaluated based on their impact on performance metrics, such as the Akaike Information Criterion (AIC). Features that significantly improve model fit and predictive power are retained, while those with minimal or redundant contributions are excluded. This process reduces the initial feature set to 20 impactful features, which are detailed in Table VII.

Given the class imbalance, we explore two sampling strategies during training: **(i)** Random undersampling, where the majority class is reduced to match the minority class, and **(ii)** Oversampling using the Synthetic Minority Oversampling Technique (SMOTE) [56], which generates synthetic samples

for the minority class.

Step 5: LLM baseline (prompt-based classification). As a baseline for resolution prediction, we also prompt a general-purpose LLM, specifically Mixtral 8x7B [20]. In a zero-shot setting, we provide only the initial question to the LLM and ask it to predict whether the question is likely to be "resolved" or "unresolved". We then compare these baseline predictions (derived solely from the initial question) to the ground-truth resolution labels, which are obtained via an LLM-based method that has access to the *entire* conversation. An example of the prompt is shown below.

Prompt: You are an AI assistant who helps analyze software engineering chatroom discussions. Your task is to determine whether the following developer question will likely receive a resolved answer based on its clarity, completeness, and technical details.
Instructions:
- Read the developer question carefully.
- Predict whether the question will be resolved (*Yes*) or not resolved (*No*).
- Do not provide an explanation—only return the classification.

Input Format:
Question: I am using TensorFlow version 2.5 and encountering an error during installation. How can I fix this?
Output Format (Strict JSON format):
{"resolution_status": "Yes"}

Step 6: Model evaluation. The trained mixed-effect model is evaluated using the *Area Under the Curve (AUC)*, which measures the model's ability to distinguish between resolved and unresolved questions. To ensure a stable evaluation, we use the following approaches: (i) 5-Fold Cross-Validation (CV), where the dataset is split into 5 folds, and each fold is used as a test set once; (ii) 10-Fold Cross-Validation (CV), which follows the same approach except with 10 folds; and (iii) Bootstrapping (100 iterations), where the dataset is repeatedly resampled with replacement to capture variability. Each evaluation method is applied across both sampling strategies (random undersampling and SMOTE).

Step 7: Feature importance analysis. We conduct a feature importance analysis to identify which features have significant positive or negative impacts on the likelihood of resolution. We examine the coefficients from both the final model and the intermediary models at each step of the feature selection process. By analyzing the changes in coefficients as new features are added, we assess how each feature contributes to the model incrementally. Positive coefficients indicate features that increase the likelihood of resolution, while negative coefficients indicate those that decrease it. This stepwise examination ensures that the final set of features is robust and their impact remains consistent throughout the process.

To assess the statistical significance of these coefficients, we examine the associated z-values and p-values. The z-value measures how many standard deviations the coefficient is from zero under the null hypothesis (H_0: *The feature does not affect the likelihood of question resolution*), with higher z-values indicating stronger evidence against the null hypothesis. Features with $p < 0.05$ are considered statistically significant.

TABLE IV
PERFORMANCE METRICS FOR ENTITY RECOGNITION, INTENT IDENTIFICATION, AND RESOLUTION STATUS DETECTION.

Metric	Entity (%)	Intent (%)	Resolution (%)
Accuracy	91	76	93
Precision	88	72	96
Recall	85	70	87
F-Score	86	71	89

G. Statistical Analysis of Entities and Intents (RQ3)

We study the interaction among software-specific entities, intents, and the resolution of developer questions through the following aspects:

Intent success rate evaluation. To understand the overall efficacy of different question types in eliciting responses, we assess the success rate for each intent by calculating the percentage of resolved questions within each intent category. We investigate if there is a significant relationship between the intent of the question and its resolution status. To evaluate this, we use a Chi-Square test of independence [57], which determines if there is an association between two categorical variables. The test compares the observed frequencies (the actual number of resolved and unresolved questions per intent) to the expected frequencies (what would be expected if there is no relationship). A large Chi-Square statistic indicates that the observed and expected values differ significantly, suggesting a relationship between the variables. Conversely, a small Chi-Square statistic suggests little or no relationship. The significance of the test is evaluated by the p-value, where a small p-value (typically less than 0.05) indicates that the observed relationship is unlikely due to chance.

Analysis of entity pairs. We analyze entity pairs, defined as two entities appearing together in the initial question of a conversation. For questions involving three or more entities, all possible pairs are considered. For example, if a question contains the entities `Device`, `Application`, and `Library`, the resulting pairs are (`Device`, `Application`), (`Device`, `Library`), and (`Application`, `Library`). This analysis is motivated by our observation that around 75% of conversations involve at least two entities. The goal of this is to understand how these relationships contribute to question resolution. In addition, we examine the resolution success rates of entity pairs across different intent categories in their effectiveness. To ensure reliable results, we filter out pairs with fewer than 10 occurrences.

III. RESULTS

In this section, we provide the findings for each of our research questions.

A. RQ1: How effective is SENIR in labelling developer chatroom conversations?

SENIR can correctly label 91% of entities. Table IV summarizes the performance metrics for each task, SENIR achieves strong performance in labelling resolution status with an F-score of 89%. For the NER and intent detection tasks, SENIR also performs well with F-scores of 86% and 71%,

TABLE V
SENSITIVITY OF INTENT DETECTION TO THE NUMBER OF INITIAL TURNS (k) INCLUDED IN THE PROMPT.

Turns included	Precision (%)	Recall (%)	F1 (%)
k=1	72	70	71
k=2	71	70	70
k=3	68	65	66
k=4	66	61	63
k=5	62	58	60

respectively. Although the recall values are slightly lower, the precision and accuracy metrics remain robust, which indicates that SENIR can accurately capture the software-specific entities and intents in developer conversations. Per-class performance for each intent and entity can be found in Appendix B.

Impact of including additional conversation turns on intent detection. To assess whether limiting SENIR's input to only the initial question constrains its ability to capture the conversation's underlying purpose, we conducted a sensitivity analysis varying the number of turns (k) provided to the model. For each conversation, we take the initial message plus the subsequent $k-1$ replies ($k \in \{1,2,3,4,5\}$) and reran the intent detection task under the same experimental conditions as in the main setup. Table V shows the resulting Precision, Recall, and F-scores (macro-averaged across intents). Our sensitivity analysis shows that when $k = 2$, performance is almost the same as $k = 1$. This is because the second message is often a clarifying question, which shifts the focus from the main question to something unclear for the second user. From $k = 3$ onward, extra replies mostly add noise (side talk or off-topic comments), so both precision and recall drop. For example, F1 decreases from 71% at $k = 1$ to 66% at $k = 3$, and further down to 60% at $k = 5$.

These findings suggest that most useful intent cues are present in the first question itself. Adding later turns does not improve accuracy and can even hurt it. Therefore, we keep $k = 1$ in the main experiments, which also helps keep the prompts short and efficient.

SENIR outperforms supervised baselines for both NER and intent detection. While SoftNER can recognize some frequent and lexical categories, in this off-the-shelf setting, SoftNER achieves 57% F1, which is lower than 86% F1 for SENIR. This performance difference is consistent with the domain shift from Stack Overflow to informal, multi-turn, fragmented chat with noises, such as abbreviations, inline code, and typos. Even when retrained on our golden dataset, SoftNER's F-score improves to 62%, but still remains well below SENIR's 86% F1 while requiring task-specific labelled data and model retraining. These results reinforce our claim that traditional sequence-labelling models struggle in chatroom settings. For intent detection, the fine-tuned BERT achieves a 54% F-score, compared to 71% for SENIR. The lower performance of these baselines highlights the challenge of applying supervised models to informal chatroom conversations where large-scale in-domain labeled data is unavailable.

Our LLM-based approach shows strong performance for concrete entities (e.g., `Error Name`, `Library Function`), but lower performance for abstract ones (e.g., `Application`). Concrete entities, such as `Error Name` (94% accuracy) and `Library Function` entities (88% accuracy), are easier to detect likely due to their well-defined context. In contrast, abstract terms like `Application` achieve a lower accuracy of 81%. These results suggest that entities grounded in concrete, well-defined terms are easier for the model to identify compared to abstract concepts. This observation aligns with previous findings by Ye et al. [11], where the baseline approach also performs well on programming languages but struggles with more nuanced categories such as the "API category".

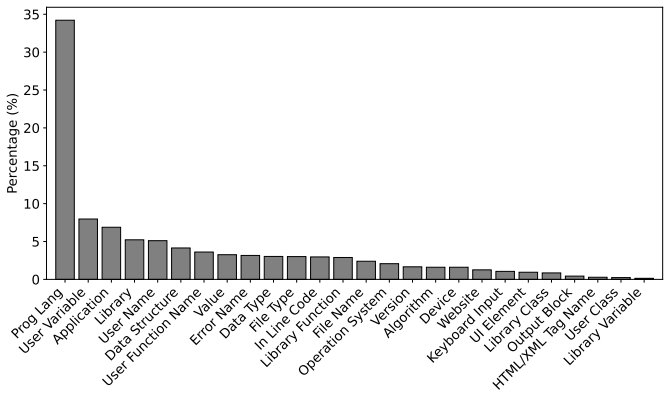

Fig. 2. Distribution of SENIR-labelled entities in the DISCO dataset.

Entities related to structured data (e.g., `Version` and `File Type`) achieve high precision and recall. Entities that represent structured information, such as `Version` and `File Type`, consistently rank the highest in terms of precision and recall. In contrast, entities such as `Library` and `In Line Code` show lower performance, which indicates that these categories may be more ambiguous or challenging for the model to detect in informal chatroom conversations.

`Programming Language` is the most frequently recognized entity in developer conversations. Fig. 2 shows the distribution of recognized entities, with `Programming Language` appearing in 34% of the conversations. This finding highlights the centrality of discussions surrounding programming languages in developer chatrooms. `User Variable` and `Application` are the next most frequent entities, indicating a significant focus on user-defined variables and software applications.

Effect of removing channel keywords. Because two channels (e.g., `python-general`, `golang`) dominate our dataset, we repeated the NER step after removing the channel keyword from every message (e.g., stripping "python" or "golang"). After re-running the pipeline, the share of `Programming Language` decreases from 34% to 22%. It remains the most frequent entity, but its dominance is reduced, indicating that channel naming inflates this count while genuine programming language mentions still occur frequently in developer chatrooms.

Intent detection shows varied performance, indicating differing levels of contextual complexity among intents. Table IV shows that certain intents, such as *Review* and

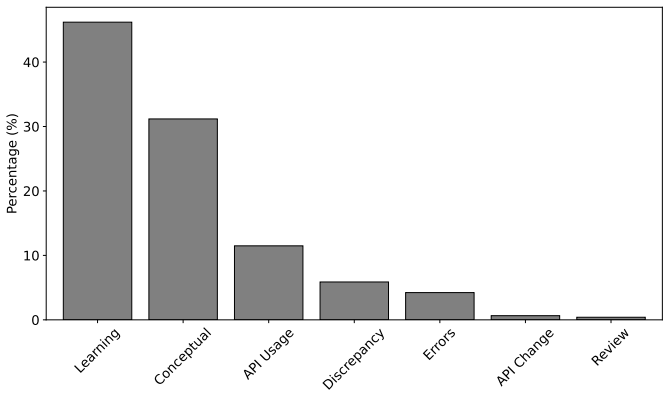

Fig. 3. Distribution of SENIR-labelled intents in the DISCO dataset.

Errors, achieve high accuracy (89%), compared to more abstract intents, such as *Conceptual* (57% accuracy). This discrepancy suggests that intents that require deep contextual understanding are more challenging for the model to capture accurately. In some instances, errors occur when entities or intents are used in unexpected contexts. For example, the term 'Go' is misclassified in discussions about travel rather than programming. This highlights a potential limitation in the model's ability to disambiguate between terms with multiple interpretations, particularly in casual or off-topic conversations.

Developer conversations predominantly focus on learning new concepts and understanding complex ideas. Fig. 3 shows that *Learning* and *Conceptual* are the most common intents in the dataset, representing 46% and 31% of all conversations. This reflects the focus of technical discussions on technical education and conceptual exploration. In contrast, *API Usage*, the third most frequent intent (11%) highlights the need for clarifications or advice on API functionalities.

Our LLM-based labelling approach shows strong agreement with manual labels. Our approach achieves Cohen's Kappa scores of 0.84 for entity recognition, 0.78 for intent detection, and 0.86 for resolution status. These scores fall within the "substantial" (0.61–0.80) and the "perfect" (0.81–1.00) agreement [58], indicating that SENIR reliably captures key aspects of conversations. However, instances of disagreement between annotators and the model often arise due to conversational ambiguity. Ambiguous cases (e.g., *Learning* vs. *API Usage*) are documented with examples in Appendix C. Disagreement also occurs when users use vague terminology in a particular development community. These challenges highlight the difficulty of intent detection in casual, context-shifting environments like developer chatrooms.

TABLE VI
AUC VALUES OF THE MIXED-EFFECT MODELS FOR THE DIFFERENT CONFIGURATIONS

Configuration	5-Fold CV	10-Fold CV	Bootstrap
Random Undersampling	0.7544	0.7545	0.7531
Oversampling with SMOTE	0.7560	0.7558	0.7546

TABLE VII
MIXED-EFFECT MODEL RESULTS FOR RESOLUTION PREDICTION

Feature	Coef.	Std.Err.	z	P>|z|	Sign.	Rel.
Sentiment	0.705	0.044	15.912	0.000	***	↗
User Name	0.097	0.017	5.745	0.000	***	↗
URLs Count	-0.298	0.077	-3.846	0.000	***	↘
Application	-0.033	0.015	-2.265	0.024	*	↘
Library	-0.062	0.018	-3.472	0.001	**	↘
Daytime	-0.020	0.014	-1.475	0.140		↘
Library Function	0.065	0.022	2.931	0.003	**	↗
Weekday	0.018	0.012	1.482	0.138		↗
UI Element	-0.061	0.043	-1.415	0.157		↘
Code Snippets	-0.380	0.103	-3.705	0.000	***	↘
Data Type	0.036	0.021	1.707	0.088	.	↗
Data Structure	0.042	0.018	2.350	0.019	*	↗
Readability CLI	-0.347	0.216	-1.605	0.109		↘
File Type	-0.42	0.021	-1.941	0.052	.	↘
Keyboard Input	0.84	0.037	2.241	0.025	*	↗
Library Class	0.86	0.040	2.154	0.031	*	↗
Question Length	-0.124	0.072	-1.716	0.086	.	↘
Organization	-0.167	0.083	-2.012	0.044	*	↘
Specific Intent Presence	-0.115	0.016	-6.976	0.000	***	↘
HTML/XML Tag Name	0.124	0.069	1.782	0.075	.	↗

Summary of RQ1

SENIR demonstrates high effectiveness in analyzing developer chatroom conversations by accurately identifying software-specific entities, intents, and resolution statuses, with an F-score of 86% for entity recognition and 89% for resolution status classification. Despite these strengths, challenges remain in detecting certain intents requiring deep contextual understanding, particularly for abstract concepts, highlighting areas for future improvement.

B. RQ2: What features of the developer questions contribute to their resolution outcomes?

The model demonstrates acceptable discrimination performance across all configurations, with AUC values consistently falling within the range of 0.7 to 0.8. Table VI summarizes the AUC values for the tested configurations, which include two sampling strategies (random undersampling and oversampling with SMOTE) evaluated using three methodologies: 5-Fold CV, 10-Fold CV, and Bootstrapping. Oversampling with SMOTE has a slight advantage and achieves the highest AUC values across all evaluation approaches, with an AUC of 0.7560 in 5-Fold CV and 0.7558 in 10-Fold CV.

Minority-class performance analysis. Since our dataset is heavily imbalanced (*unresolved*: 77.95% vs. *resolved*: 22.05%), we further evaluate the model's ability to identify resolved cases. In addition to AUC, we compute precision, recall, and F1 scores for the resolved class. Under our best-performing configuration (random undersampling with 5-Fold

CV), the resolved class achieves an F1 of 48%, with a precision of 34% and a recall of 84%. This high recall indicates that the model is well-suited for identifying the vast majority of questions that have the potential to be resolved. However, the lower precision suggests a trade-off where the model may be over-optimistic, flagging some questions as resolvable that ultimately remain unanswered. Consequently, tools leveraging this model for automated feedback can effectively identify "promising" questions but should account for this tendency to over-suggest improvements. This shows the model is not only good at predicting "unresolved" but can also find resolvable questions, even with the class imbalance.

Impact of imbalance-handling strategies. To better understand how undersampling and oversampling affect results, we compare both approaches in detail. Random undersampling improves recall for the minority *resolved* class (up to 0.84) but at the cost of lower precision (0.33), reflecting its tendency to aggressively balance the dataset by discarding unresolved cases. In contrast, SMOTE preserves stable AUC values (around 0.75) but yields low recall for the resolved class, indicating that synthetic examples do not sufficiently generalize in this setting. These results show that undersampling is better suited for highlighting resolvable questions, while oversampling maintains dataset size but risks overfitting. Accordingly, our subsequent feature-importance analysis is based on the undersampling configuration, which offers the most balanced view of minority-class performance.

The stable results across all configurations provide confidence in the model's ability to distinguish between resolved and unresolved questions, allowing us to proceed with analyzing feature importance to understand the factors contributing to resolution outcomes.

Furthermore, when comparing these results to the LLM baseline, treated as a plain Yes/No classifier, it achieves an accuracy of 49% and an ROC-AUC of 50%. This is below the performance of the feature-based mixed-effect model reported above, confirming that simple zero-shot prompting on the initial question is not competitive for resolution prediction in our setting.

While certain SE entities provide helpful context to drive resolution, others may introduce unnecessary complexity or ambiguity. The subset of features representing SE entities (denoted in capital letters) provides insights into how the presence of specific entities influences question resolution, as shown in Table VII. These features are binary, indicating whether a particular entity is present (1) or absent (0) in a question. Among these, `Application` (e.g., "Flask," "PyCharm"), `Library` (e.g., "NumPy," "React"), and Code Snippets have a significant negative effect on resolution likelihood ($p < 0.05$). For instance, questions that include Code Snippets are negatively correlated with resolution ($z = -3.705$, coefficient $= -0.380$), potentially indicating that such questions might introduce complexity or require additional context that impedes resolution. Similarly, the presence of `Application` and `Library` entities appears to slightly detract from resolution outcomes, possibly due to their broad or ambiguous nature. In contrast, entities like `Library Function` (e.g., "numpy.mean()," "json.dumps()") and `Library Class` (e.g., "DataFrame," "Button") demonstrate a positive correlation with resolution ($p < 0.05$), suggesting that including specific, well-defined technical elements increases the likelihood of resolution. For example, the feature `Library Function` has a coefficient of 0.065 ($z = 2.931$), indicating a modest but significant positive relationship. This finding highlights the value of technical specificity in driving resolution success.

Interpretation with examples. Beyond the statistical coefficients, we examine the mechanisms explaining why certain entities affect resolution. Specific entities such as `Library Function` and `Library Class` provide focused technical context that narrows the answerer's search space and reduces ambiguity. In contrast, broad entities like `Application` or generic `Library` mentions are often ambiguous and lead to clarification loops. Similarly, long `Code Snippets` and excessive `URLs` may slow resolution by adding noise and shifting cognitive load to the responder. The social dimension is also important, positive sentiment and user tagging increase engagement and accountability, which aligns with the AUC improvement when early-reply features are added (from 0.75 to 0.86). Detailed examples are provided in Appendix C.

Sensitivity analysis with early-reply features. To test for possible omitted-variable bias, we extended the mixed-effect model by adding early-reply features such as *time to first reply*, *first-reply length*, *first-reply sentiment*, and *first-reply code snippet*. The updated model achieved a higher AUC of **0.866**, compared to the original model's AUC of **0.754**. This improvement confirms that early-reply factors significantly influence whether a question is resolved.

Effect of removing channel keywords. We re-train the mixed-effect model on the channel-keyword-stripped dataset. In the original model, the `Programming Language` feature had a small positive coefficient, but in the stripped dataset its effect is lost and removed during the AIC-based stepwise selection.

Other features with a positive impact include Sentiment, where a more positive tone significantly improves resolution likelihood, and User Name, indicating that directly mentioning specific users could improve responsiveness. This result confirms the findings by Ehsan et al. [4]. On the other hand, URLs Count has a negative effect on resolution, indicating that later excessive URLs may introduce complexity or reduce focus, making questions harder to address effectively.

Summary of RQ2

The mixed-effect model demonstrates acceptable performance in distinguishing resolved and unresolved questions, with stable AUC values across all configurations (0.7 to 0.8). Feature importance analysis highlights that while specific SE entities (e.g., `Library Function`, `Library Class`) and positive sentiment improve resolution likelihood, elements like excessive URLs, and entity mentions detract from resolution.

TABLE VIII
RESOLUTION OUTCOME BY INTENT

Intent	% Success	# Success	# Total
API Usage	33.6	1,845	5,497
API Change	26.2	81	309
Errors	25.6	519	2,024
Conceptual	23.7	3,535	14,924
Learning	22.9	5,053	22,112
Discrepancy	22.0	618	2,813
Review	18.8	36	192
Overall	21.9	6,412	29,243

C. RQ3: How do entities, intents, and their interactions impact the resolution of developer questions?

The success rates for the different intents vary considerably from 18.8% to 33.6%, highlighting an overall suboptimal rate of resolution across all intents. Table VIII shows that *API Usage* (33.6%) and *API Change* (26.2%) have the highest success rates, while *Discrepancy* (22%) and *Review* (18.8%) are on the lower end. This points to potential difficulties in addressing more complex or nuanced questions. To further assess the relationship between intent and resolution status, a Chi-Square test is conducted. The test yields a Chi-Square statistic of 76.83 with 6 degrees of freedom and a p-value of $1.61e^{-14}$. The large Chi-Square statistic, far exceeding the critical threshold, and the small p-value strongly indicate that the differences in resolution rates across intents are statistically significant. This suggests that certain intents are more likely to result in successful resolution than others, reinforcing the need for better question formulation in lower-performing categories like *Discrepancy* and *Review*.

Entity pair analysis reveals that certain combinations are more effective in driving question resolution under specific intents. As shown in Table IX, for the *API Change* intent, the pair (`Data Structure`, `Output Block`) achieves a success rate of 75%, On the other hand, the pair (`File Type`, `Keyboard Input`) yields 17.7% suggesting these entities might be less useful in eliciting responses.

For the *API Usage* intent, the pair (`Application`, `File Type`) has a success rate of 53.3%, showing that providing specific application-related and file-related information is beneficial. However, combinations involving the pair (`Value`, `Keyboard Input`) yield only 5.6%, indicating their ineffectiveness in this context.

In the *Learning* intent, the pair (`Programming Language`, `Library`) has a success rate of 53.9%, while combinations involving the pair (`Programming Language`, `User Name`) result in a 0% success rate, suggesting that providing detailed library information is much more beneficial than including user-specific information.

Regarding the *Discrepancy* intent, we observe a shift towards diagnostic-focused entities such as `Library Function` and `User Variable`. However, combinations involving the pair (`UI Element`, `File Name`) show lower success rates, indicating that backend-related information tends to be more effective for resolution.

Similarly, the *Review* intent tends to occur with entities related to detailed codebase elements, such as `Library Variable`, `Version` and `UI Element`, indicating that these conversations often involve reviewing or resolving problems within specific parts of the codebase. Our observations show that the SENIR-labelled entities accurately capture the technical content of the questions and reflect the associated intents, as initially intended.

TABLE IX
ENTITY PAIR RESULTS (TOP 3 AND BOTTOM 3 BY SUCCESS RATE).

Intent	Entity Pair	% Succ	# Total
Aggregate Results (All Intents)	`Device, Library Variable`	45.5	11
	`Version, Library Variable`	41.7	12
	`Data Type, Library Variable`	36.4	22
	`Application, User Class`	10.0	20
	`User Variable, UI Element`	8.4	154
	`User Class, Value`	6.9	29
The following rows present entity pair results broken down by intent.			
API Usage	`Application, File Type`	53.3	15
	`Operation System, User Variable`	45.8	24
	`Data Structure, Library Function`	45.1	51
	`Value, Keyboard Input`	5.6	18
	`User Func Name, Keyboard Input`	5.0	20
	`Value, Output Block`	0.0	10
Conceptual	`Data Structure, Library Variable`	42.1	19
	`User Name, HTML/XML Tag Name`	41.7	12
	`Version, Library Variable`	41.7	12
	`User Func Name, Website`	6.9	29
	`Algorithm, UI Element`	6.3	16
	`User Variable, Website`	5.6	36
Discrepancy	`User Variable, Library Function`	39.1	115
	`Library Class, Library Function`	38.7	31
	`Application, File Type`	38.5	26
	`Data Type, Keyboard Input`	0.0	10
	`File Name, UI Element`	0.0	11
	`User Func Name, Keyboard Input`	0.0	21
Errors	`Prog Lang, Library Function`	63.6	11
	`Prog Lang, File Name`	41.2	17
	`Prog Lang, File Type`	33.3	12
	`Prog Lang, Error Name`	27.3	11
	`Prog Lang, Version`	14.3	14
	`Prog Lang, User Name`	0.0	10
Learning	`Prog Lang, Library`	53.9	13
	`User Variable, User Func Name`	50.0	10
	`Prog Lang, User Func Name`	46.2	13
	`Prog Lang, Application`	15.4	13
	`Prog Lang, Website`	10.0	10
	`Prog Lang, User Name`	0.0	10
Review	`Device, Library Variable`	40.0	10
	`Version, Data Structure`	37.5	64
	`UI Element, HTML/XML Tag Name`	36.8	38
	`File Name, Website`	10.0	90
	`Application, User Func Name`	9.1	154
	`User Variable, UI Element`	8.9	124
API Change	`Data Structure, Output Block`	75.0	12
	`Algorithm, Error Name`	66.7	12
	`Data Type, UI Element`	63.6	11
	`File Type, Keyboard Input`	17.7	34
	`Library, Website`	17.1	35
	`Application, User Func Name`	17.0	59

Interpretation with examples by intent. Our entity-pair analysis reveals that the effectiveness of entity combinations varies by developer intent. Pairs that include clear diagnostic information (e.g., `Library Function` + `User Variable` under "Error" intent) help more because they show both the problem and where the change happens, supporting clear fault localization. In contrast, pairs that are too

general or only mention UI components (e.g., `UI Element` + `File Name` under "Discrepancy" intent) often work less well because they lack the technical depth needed for actionable responses Illustrative examples for each intent are provided in Appendix C.

Effect of removing channel keywords. We also repeat the entity-intent pair analysis on the stripped dataset. Pairs involving `Programming Language` still appear among the top three for intents, such as *Learning* and *Errors*, but their frequency and margins are smaller than in the unstripped version. For example, in the *Learning* intent, the top three successful entity pairs are: (`User Variable`, `User Function Name`) 50.0%, (`Programming Language`, `Library`) 48.9%, and (`Application`, `Operating System`) 47.4%. For the *Errors* intent, the leading pairs were: (`Programming Language`, `Library Function`) 66.1%, (`Programming Language`, `File Name`) 43.9%, and (`Programming Language`, `File Type`) 38.8%.

Summary of RQ3

The analysis in RQ3 reveals that specific combinations of software-specific entities and intents impact the likelihood of question resolution. Dominant entities such as `Programming Language` and `Library` play a key role across multiple intents, while entities like `User Variable` and `UI Element` are less beneficial. Success rates vary considerably among intents, with *API Usage* and *API Change* showing the highest resolution rates, while *Discrepancy* and *Review* show lower rates. The Chi-Square analysis confirms that resolution rates significantly differ across intents, suggesting that better question formulation is needed for lower-performing categories.

IV. Implications

In this section, we discuss our findings and their possible implications for developers, chatroom platforms, and researchers.

A. Implications for Developers

It is important to provide focused questions with specific technical details. Our findings in Sections III-B and III-C reveal that including precise entities, such as `Library Function` or `Library Class`, correlates with higher resolution rates, particularly for intents like *Discrepancy* and *Errors*. Rather than relying on large `CODE SNIPPETS` or abstract references (e.g., `Application`), developers should pinpoint the exact function or class in question.

It is valuable to maintain a positive tone, tag specific users, and avoid overloading with URLs. As highlighted in Section III-B, questions formulated with a positive tone are more likely to be resolved. Likewise, tagging specific users (e.g., "@UserName"), when appropriate, increases visibility and may improve the response rate. However, including too many URLs can overload the conversation and detract from the core question, which can potentially slow down or prevent the resolution.

B. Implications for Chatroom Platforms

Chatroom platforms can benefit from offering structured question templates. Questions that specify concrete entities and intents correlate with higher resolution rates (Section III-C). Chatroom platforms can leverage these insights by providing structured templates that prompt users to include important technical details. For example, if a user indicates they are dealing with an error, the platform could prompt them to specify the `Programming Language`, `Library`, and relevant `Library Function`, to help minimize ambiguity of the questions.

Chatroom platforms can benefit from integrating an approach like SENIR for automated tagging and highlighting. As shown in Section III-A, SENIR effectively labels software-specific entities and intents. Chatroom platforms could integrate SENIR to automatically generate tags for new questions to draw attention to key entities (e.g., `Programming Language`) and identify intents (e.g., *API Usage*). While our resolution prediction model achieves high recall (detecting most resolvable questions), platforms should account for its lower precision by positioning automated feedback as suggestions rather than guarantees of resolution. This approach can enable developers to quickly assess a question's context and filter questions based on their expertise.

C. Implications for Researchers

Researchers can use SENIR to label other developer conversation datasets. Researchers can use SENIR to label other developer conversation datasets. The results in Section III-A confirm that SENIR can reliably label developer conversations with software-specific entities, intents, and resolution statuses in the context of Discord. While our study focuses on Discord, SENIR is designed to be applicable to other developer chat platforms, such as GitHub Discussions.[4] To do this, researchers may need to make some adaptations, such as updating the entity types or changing the prompt text, in order to match the style and vocabulary of the new dataset. For example, a dataset from GitHub Discussions or Stack Overflow may have different terms or formats compared to Discord. In this paper, we only tested SENIR on Discord data. Therefore, our performance evaluation results and the analysis of question resolution apply only to that platform. The SENIR method can be adapted to other datasets to support studies on software engineering communication in different communities.

Researchers can leverage SENIR to enhance chatbots in developer chatrooms. SENIR's entity, intent, and resolution labels provide valuable signals for improving chatbot-based assistance in chatrooms. For example, researchers can use the resolution status to curate high-quality resolved questions for retrieval-augmented generation (RAG) [59]. This enables chatbots to provide more reliable answers based on past conversations. In addition, entities and intents can refine

[4]https://docs.github.com/en/discussions

retrieval strategies, techniques such as GraphRAG [60] can build knowledge graphs to enhance retrieval by leveraging labelled entities and intents.

V. Related Work

In this section, we discuss related work about NER in software engineering contexts, LLM-centric NER, LLM applications in software engineering tasks, and developer chatrooms and online forums, and question-quality prediction in Q&A forums.

NER in software engineering. NER has been widely studied in the software engineering domain for automatically identifying and categorizing software-specific entities within text from various sources such as source code, commit messages, documentation, and social media content. Ye et al. [11] developed machine learning based NER systems for software engineering social content. Their approach demonstrates improved performance over rule based systems by addressing entity ambiguity and informal language. Similarly, Tabassum et al. [14] highlighted the importance of domain-specific NER for understanding code-related discussions. Their approach emphasizes the role of software-specific categories like APIs, frameworks, and libraries.

However, both Ye et al. [11] and Tabassum et al. [14] discuss the substantial feature engineering required for traditional, supervised NER methods, and highlight the difficulty these methods face when applied to noisy, domain-specific, or informal content such as developer chatrooms. This challenge limits adaptability and robustness for social platform data.

Recent advancements leverage transformer models like BERT [61] for NER, improving contextual understanding in SE texts. SoftNER [14], trained on Stack Overflow, reports strong results for software-related entity recognition in structured Q&A. Because our taxonomy partially inherits StackOverflow-centric labels, we include SoftNER as a baseline in Section III-A to test adaptability to informal, fragmented Discord conversations. Das et al. [31] further explored zero-shot NER to recognize unseen entities in low-resource settings. While these studies focus on structured platforms, our work targets the fragmented, dynamic nature of developer chatrooms (e.g., Discord), jointly modeling entities, intents, and resolution status.

LLM-centric NER. Various research approaches demonstrate that it is possible to do NER with LLMs without any fine-tuning. LTNER [62] uses GPT-3.5 by wrapping each target span in a unique marker, e.g., replacing "Python" with "<entity_type> Python </entity_type>" and providing a few annotated examples in the prompt. LTNER can produce results close to the supervised models without training the model. llmNER [63] is a library that makes zero-shot or few-shot NER using LLMs easier. llmNER provides a Python API that constructs the appropriate few-shot or zero-shot prompts, submits them to a specified LLM endpoint (e.g., OpenAI), and parses the model's JSON-formatted output back into token-level entity labels, without any model weight updates. There are also other methods, such as Xie et al. [64], which focus on iterative refinement and self-improvement, where the LLM can improve its own predictions step by step on new text, so the NER quality can get better over multiple iterations. These methods show that only prompting the LLM, without training, can still give strong NER results through various strategies including contextual entity marking [62], zero/few-shot frameworks [63], and self-improving mechanisms [64]. Our SENIR method also uses LLM prompting to get good labels for messy and informal chatroom messages.

LLM applications in software engineering. LLMs have shown significant potential in automating a broad range of SE tasks, such as code generation, bug detection, and documentation [65], [66]. Although several studies focus on retrieving Q&A content from structured platforms [67], researchers have also explored LLM-based methods for tasks like requirements engineering [68], [69] and code refinement [70]. Colavito et al. [71] further demonstrated that GPT-like models can effectively classify GitHub issues, reducing human workload in issue labelling. Our work extends LLM applications to dynamic chatroom environments by introducing automated labelling of entities and intents and leveraging the deep contextual understanding of LLMs to enhance question clarity, enabling structured analysis and actionable feedback for developers. This integration extends the applicability of LLMs to dynamic chatroom environments, where conventional retrieval methods struggle.

Developer chatrooms and online forums. Developer chatrooms (e.g., Slack, Discord) enable real-time collaboration, but their informal style complicates message analysis. Empirical studies have shown that missing details often lead to unanswered questions, whereas user mentions can boost engagement [4]. Subash et al. [19] introduced the DISCO dataset to highlight the unique challenges of disentangling and analyzing these multi-threaded discussions. Similarly, El Mezouar et al. [21] and Shi et al. [72] found that developer chatrooms contain valuable knowledge but remain difficult to study due to their fluid format.

Lill et al. [9] found that reusing past chats and Q&A posts to resolve new Discord questions is helpful in only 40% of cases partly because questions often lack clarity. Similarly, Tufano et al. [73] examined how developers use LLMs like ChatGPT to seek assistance in open-source projects, underscoring the growing influence of AI-based aids. While these studies focus on overall chatroom behaviour, our work aims to refine questions by jointly modelling NER, intent, and resolution status within a unified framework. By extracting software-specific entities and understanding the question's underlying purpose, we enable structured insights that pave the way for higher-quality discussions and faster problem resolutions.

Question-quality prediction in Q&A forums. Many researchers study how to decide if a question in Q&A sites is good or not. Anderson et al. [22] looked at how the community reacts to questions over time. They aim to predict the long-term value of a question (like page views one year later) and if a question is well answered soon after it is posted. Asaduzzaman et al. [23] collected unanswered questions on Stack Overflow and studied why they stay unanswered. They built predictors to estimate how long a question will stay unanswered. Other studies [25], [26], [24] check the quality

from different views. Correa et al. [24] predicts if a question will be closed at the time of the question creation. Baltadzhieva et al. [25] and Calefato et al. [26] check which words and details make a question more clear or get more answers. All these show that good questions are clear, specific, and have enough technical context (like code or error messages). Different from the prior work, we study chatrooms, which are not structured forums and are informal and questions evolve fast. We use software-specific signals like entities and intents, to label if a question is resolved. This helps give quick feedback to improve the quality of questions and get answers faster.

Synthesis and gap. Prior efforts on SE NER (e.g., Soft-NER) [14], zero/few-shot LLM-based NER [62], [63], [64] and question-quality prediction on structured Q&A [22], [23], [24], [25], [26] address complementary pieces of the problem. To the best of our knowledge, no prior work jointly performs software-specific NER, intent detection, and resolution status labelling on informal, multi-turn chatroom conversations. Our study targets unstructured, informal chatrooms and contributes an end-to-end, zero-shot prompting pipeline that jointly labels software-specific entities, intents and resolution, and then uses these labels to (i) train a mixed-effect predictor from the initial question alone and (ii) analyze intent-conditioned entity patterns linked to resolution. This bridges a gap between structured forum settings and dynamic chat environments.

VI. Threats to Validity

In this section, we discuss the threats to validity of our study about refining developer questions in chatrooms.

Construct Validity. We study only one LLM, Mixtral, due to its larger token size compared to other open-source models. However, this may limit the generalizability of our results to other models. To mitigate this threat, we carefully select a set of software-specific entities and intent categories that are widely recognized within the software development community. Two PhD students with expertise in software engineering and natural language processing manually label the dataset, ensuring that the entity and intent classification schemes accurately reflect real-world scenarios.

Internal Validity. The primary threat is potential bias in manual labelling and the influence of specific prompt designs on the results. To address this, we employ a rigorous labelling process with two annotators and calculate the inter-annotator reliability using Cohen's Kappa to reduce subjective bias. Moreover, we test different prompt designs in a preliminary study to select the most effective ones for our main experiments.

External Validity. Since our study focuses on Discord chatrooms, the results may not apply to other platforms, such as Stack Overflow and GitHub Discussions, which may have different writing styles and community rules. To mitigate this threat, we analyze a large dataset spanning multiple software engineering-related chatrooms to ensure our findings are not specific to a single community. We also select chatrooms with diverse programming topics to improve the applicability of our results. While this limitation affects the generalizability of our results, it does not constrain the SENIR method itself, which can be adapted to other platforms by modifying the entity schema or prompt wording (Section IV). While we acknowledge the diversity in software development communities, we encourage future work to test SENIR on more datasets to confirm its performance in other settings.

TABLE X
Post-LLM developer engagement trends in our target Discord channels: comparing a pre-LLM window (2019–2020) with a recent post-LLM window (2024–2025).

Python channel			
Metric	**2019–2020**	**2024–2025**	**Change**
total_messages_in_year	591,181	1,342,101	127%↑
total_users_in_year	9,214	16,462	79%↑
avg_messages_per_day	1,615	3,677	128%↑
avg_messages_per_week	11,154	25,323	127%↑
avg_users_per_month	1,149	2,162	88%↑
avg_users_per_week	384	729	90%↑

Temporal Validity. Our main dataset (DISCO) ends in 2022, before the widespread adoption of LLMs. To ensure developers still engage in chatrooms after the launch of LLMs, we examined developer activity in public community channels during the post-LLM period. A large-scale Discord analysis, covering the period of 2015–2024, reports 2.05 billion messages from 4.74 million users across 3,167 servers, confirming sustained activity through 2024 [74]. Consistent with these findings, metadata from the same Discord communities we have studied reveal substantial growth in both message volume and unique participation between a pre-LLM window (2019–2020) and a post-LLM window (2024–2025). As shown in Table X, we observe a 127% increase in message volume and 79% increase in users.

To check for post-LLM drift, we run a subset of the 2024–2025 period from the same servers (around 500 conversations). We re-apply SENIR and re-estimate the mixed-effect model on this subset using the same feature set as in RQ2. The *direction* of the most influential coefficients remained stable (e.g., positive for `Library Function` and `Library Class`, negative for `Application`, `URLs`, and `Code Snippets`.

RQ2 and RQ3 rely on SENIR-generated resolution labels for large-scale analysis, which introduces dependency between the labelling and downstream modeling. While RQ1 validates these labels against the human-labeled golden set, achieving an 89% F1 score for resolution classification (Table IV, Section III-A), full-scale human annotation of 29k conversations is infeasible. To mitigate this threat, we use zero-shot prompting based solely on entity and intent definitions, avoiding any direct reference to the golden set and thereby preventing data leakage. Moreover, RQ2's mixed-effects model uses only features from the initial question (Section III-B), which are distinct from the conversation-level cues used by SENIR, further reducing the risk of label leakage.

VII. Conclusion

In this study, we present SENIR, an LLM-based approach for labelling chatroom conversations with software-specific named entities, intents, and resolution outcomes to understand

and refine developer questions. Through the lens of three research questions, we demonstrate how these structured labels deepen our understanding of developer Q&A in chatrooms and guide improvements in question formulation. Our experiments on 29,243 conversations from the DISCO dataset showed SENIR's robust performance for entity extraction (**86% F-score**), intent detection (**71% F-score**), and resolution status classification (**89% F-score**). Leveraging these labels, we built predictive models of conversation resolution and found that certain entity-intent combinations (e.g., `Library Function` with *Errors*) increase success, while features like excessive URLs and late posting times hinder resolution. A Chi-Square analysis further confirmed significant differences in resolution rates across various intents, suggesting actionable paths for refining developer questions. Future research can build on our study by exploring real-time feedback mechanisms or extending the approach to additional developer support communities, thereby shaping more targeted, efficient, and high-resolution Q&A.

ACKNOWLEDGMENTS

We want to express our appreciation to Bihui Jin at the University of Waterloo for helping to manually validate our results in order to build the golden dataset. His commitment was really helpful in confirming the reliability and precision of our findings.

REFERENCES

[1] M.-A. Storey, C. Treude, A. Van Deursen, and L.-T. Cheng, "The impact of social media on software engineering practices and tools," in *Proceedings of the FSE/SDP workshop on Future of software engineering research*, 2010, pp. 359–364.

[2] B. Vasilescu, D. Posnett, B. Ray, M. G. van den Brand, A. Serebrenik, P. Devanbu, and V. Filkov, "Gender and tenure diversity in GitHub teams," in *Proceedings of the 33rd annual ACM conference on human factors in computing systems*, 2015, pp. 3789–3798.

[3] J. Tsay, L. Dabbish, and J. Herbsleb, "Let's talk about it: evaluating contributions through discussion in GitHub," in *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, 2014, pp. 144–154.

[4] O. Ehsan, S. Hassan, M. E. Mezouar, and Y. Zou, "An empirical study of developer discussions in the Gitter platform," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 1, pp. 1–39, 2020.

[5] U. Arora, N. Goyal, A. Goel, N. Sachdeva, and P. Kumaraguru, "Ask it right! identifying low-quality questions on community question answering services," in *2022 International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8.

[6] M. Liu, X. Peng, Q. Jiang, A. Marcus, J. Yang, and W. Zhao, "Searching StackOverflow questions with multi-faceted categorization," in *Proceedings of the 10th Asia-Pacific Symposium on Internetware*, 2018, pp. 1–10.

[7] S. Beyer, C. Macho, M. Pinzger, and M. Di Penta, "Automatically classifying posts into question categories on Stack Overflow," in *Proceedings of the 26th Conference on Program Comprehension*, 2018, pp. 211–221.

[8] J. D. M.-W. C. Kenton and L. K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of naacL-HLT*, vol. 1. Minneapolis, Minnesota, 2019, p. 2.

[9] A. Lill, A. N. Meyer, and T. Fritz, "On the helpfulness of answering developer questions on Discord with similar conversations and posts from the past," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.

[10] J. Li, A. Sun, J. Han, and C. Li, "A Survey on Deep Learning for Named Entity Recognition," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 50–70, 2022.

[11] D. Ye, Z. Xing, C. Y. Foo, Z. Q. Ang, J. Li, and N. Kapre, "Software-specific named entity recognition in software engineering social content," in *IEEE 23rd international conference on software analysis, evolution, and reengineering (SANER)*, vol. 1, 2016, pp. 90–101.

[12] L. Ratinov and D. Roth, "Design challenges and misconceptions in named entity recognition," in *Proceedings of the thirteenth conference on computational natural language learning (CoNLL-2009)*, 2009, pp. 147–155.

[13] X. Liu, S. Zhang, F. Wei, and M. Zhou, "Recognizing named entities in tweets," in *Proceedings of the 49th annual meeting of the association for computational linguistics: human language technologies*, 2011, pp. 359–367.

[14] J. Tabassum, M. Maddela, W. Xu, and A. Ritter, "Code and named entity recognition in StackOverflow," *arXiv:2005.01634*, 2020.

[15] T. Brown, B. Mann *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[16] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, vol. 1, no. 2, 2023.

[17] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[18] Q. Huang, X. Xia, D. Lo, and G. C. Murphy, "Automating Intention Mining," *IEEE Transactions on Software Engineering*, vol. 46, no. 10, pp. 1098–1119, 2020.

[19] K. M. Subash, L. P. Kumar, S. L. Vadlamani, P. Chatterjee, and O. Baysal, "DISCO: A dataset of Discord chat conversations for software engineering research," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 227–231.

[20] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv:2401.04088*, 2024.

[21] M. El Mezouar, D. A. da Costa, D. M. German, and Y. Zou, "Exploring the use of chatrooms by developers: an empirical study on Slack and Gitter," *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 3988–4001, 2021.

[22] A. Anderson, D. Huttenlocher, J. Kleinberg, and J. Leskovec, "Discovering value from community activity on focused question answering sites: a case study of Stack Overflow," in *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2012, pp. 850–858.

[23] M. Asaduzzaman, A. S. Mashiyat, C. K. Roy, and K. A. Schneider, "Answering questions about unanswered questions of stack overflow," in *10th Working Conference on Mining Software Repositories (MSR)*, 2013, pp. 97–100.

[24] D. Correa and A. Sureka, "Fit or unfit: analysis and prediction of 'closed questions' on stack overflow," in *Proceedings of the first ACM conference on Online social networks*, 2013, pp. 201–212.

[25] A. Baltadzhieva and G. Chrupała, "Predicting the quality of questions on stackoverflow," in *Proceedings of the international conference recent advances in natural language processing*, 2015, pp. 32–40.

[26] F. Calefato, F. Lanubile, and N. Novielli, "How to ask for technical help? Evidence-based guidelines for writing questions on Stack Overflow," *Information and software technology*, vol. 94, pp. 186–207, 2018.

[27] P. Fathollahzadeh, M. El Mezouar, H. Li, Y. Zou, and A. E. Hassan, "Replication Package," https://github.com/Software-Evolution-Analytics-Lab-SEAL/TSE2025-SENIR, 2025.

[28] G. D. Israel, "Determining sample size," 1992.

[29] S. Noei, H. Li, S. Georgiou, and Y. Zou, "An empirical study of refactoring rhythms and tactics in the software development process," *IEEE Transactions on Software Engineering*, vol. 49, no. 12, pp. 5103–5119, 2023.

[30] J. Cohen, *Statistical power analysis for the behavioral sciences*. routledge, 2013.

[31] S. Das, N. Deb, A. Cortesi, and N. Chaki, "Zero-shot Learning for Named Entity Recognition in Software Specification Documents," in *IEEE 31st International Requirements Engineering Conference (RE)*, 2023, pp. 100–110.

[32] M. Veera Prathap Reddy, P. Prasad, M. Chikkamath, and S. Mandadi, "NERSE: named entity recognition in software engineering as a service," in *Service Research and Innovation: 7th Australian Symposium, ASSRI 2018, Sydney, NSW, Australia, September 6, 2018, and Wollongong, NSW, Australia, December 14, 2018, Revised Selected Papers 7*. Springer, 2019, pp. 65–80.

[33] C. Rosen and E. Shihab, "What are mobile developers asking about? a large scale study using Stack Overflow," *Empirical Software Engineering*, vol. 21, pp. 1192–1223, 2016.

[34] M. Allamanis and C. Sutton, "Why, when, and what: analyzing Stack Overflow questions by topic, type, and code," in *Proceedings of the 10th Working Conference on Mining Software Repositories*, 2013, pp. 53–56.

[35] C. Treude, O. Barzilay, and M.-A. Storey, "How do programmers ask and answer questions on the web? (NIER track)," in *Proceedings of the 33rd international conference on software engineering*, 2011, pp. 804–807.

[36] S. Beyer and M. Pinzger, "A manual categorization of Android app development issues on Stack Overflow," in *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 2014, pp. 531–535.

[37] S. Beyer, C. Macho, M. Di Penta, and M. Pinzger, "Analyzing the relationships between Android API classes and their references on Stack Overflow," *Univ. Klagenfurt, Klagenfurt, Austria, Tech. Rep. AAU-SERG-2017–002*, 2017.

[38] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, B. Zhu, H. Zhang, M. I. Jordan, J. E. Gonzalez, and I. Stoica, "Chatbot arena: an open platform for evaluating LLMs by human preference," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.

[39] OpenAI, "Prompt Engineering," https://platform.openai.com/docs/guides/prompt-engineering, last visited: Feb 23, 2025.

[40] Huggingface, "Chat Templates," https://huggingface.co/docs/transformers/main/en/chat_templating, last visited: Feb 23, 2025.

[41] S. Mondal, C. K. Saifullah, A. Bhattacharjee, M. M. Rahman, and C. K. Roy, "Early detection and guidelines to improve unanswered questions on Stack Overflow," in *Proceedings of the 14th Innovations in Software Engineering Conference (formerly known as India Software Engineering Conference)*, 2021, pp. 1–11.

[42] I. Srba and M. Bielikova, "Why is Stack Overflow failing? preserving sustainability in community question answering," *IEEE Software*, vol. 33, no. 4, pp. 80–89, 2016.

[43] M. Coleman and T. L. Liau, "A computer readability formula designed for machine scoring." *Journal of Applied Psychology*, vol. 60, no. 2, p. 283, 1975.

[44] X.-L. Yang, D. Lo, X. Xia, Z.-Y. Wan, and J.-L. Sun, "What security questions do developers ask? a large-scale study of Stack Overflow posts," *Journal of Computer Science and Technology*, vol. 31, pp. 910–924, 2016.

[45] B. Vasilescu, A. Serebrenik, M. Goeminne, and P. Devanbu, "How social Q&A sites are changing knowledge sharing in open source software communities," in *Proceedings of the 17th ACM conference on Computer supported cooperative work & social computing*. ACM, 2014, pp. 342–354.

[46] L. Backstrom, D. Huttenlocher, J. Kleinberg, and X. Lan, "Group formation in large social networks: membership, growth, and evolution," in *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2006, pp. 44–54.

[47] C. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 8, no. 1, 2014, pp. 216–225.

[48] E. Guzman and B. Bruegge, "Towards emotional awareness in software development teams," in *Proceedings of the 2013 9th joint meeting on foundations of software engineering*, 2013, pp. 671–674.

[49] D. J. Hand, "Assessing the performance of classification methods," *International Statistical Review*, vol. 80, no. 3, pp. 400–414, 2012.

[50] T. H. Nguyen, B. Adams, and A. E. Hassan, "Studying the impact of dependency network measures on software quality," in *2010 IEEE International Conference on Software Maintenance*. IEEE, 2010, pp. 1–10.

[51] E. Noei, F. Zhang, and Y. Zou, "Too many user-reviews! what should app developers look at first?" *IEEE Transactions on Software Engineering*, vol. 47, no. 2, pp. 367–378, 2019.

[52] S. Noei, H. Li, and Y. Zou, "Detecting Refactoring Commits in Machine Learning Python Projects: A Machine Learning-Based Approach," *arXiv:2404.06572*, 2024.

[53] J. T. Pintas, L. A. Fernandes, and A. C. B. Garcia, "Feature selection methods for text classification: a systematic literature review," *Artificial Intelligence Review*, vol. 54, no. 8, pp. 6149–6200, 2021.

[54] J. Jiarpakdee, C. Tantithamthavorn, A. Ihara, and K. Matsumoto, "A study of redundant metrics in defect prediction datasets," in *2016 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2016, pp. 51–52.

[55] Z. Zhang, "Variable selection with stepwise and best subset approaches," *Annals of translational medicine*, vol. 4, no. 7, 2016.

[56] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.

[57] M. L. McHugh, "The Chi-square test of independence," *Biochemia medica*, vol. 23, no. 2, pp. 143–149, 2013.

[58] ——, "Interrater reliability: the kappa statistic," *Biochemia medica*, vol. 22, no. 3, pp. 276–282, 2012.

[59] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474.

[60] H. Han, Y. Wang, H. Shomer, K. Guo, J. Ding, Y. Lei, M. Halappanavar, R. A. Rossi, S. Mukherjee, X. Tang, Q. He, Z. Hua, B. Long, T. Zhao, N. Shah, A. Javari, Y. Xia, and J. Tang, "Retrieval-Augmented Generation with Graphs (GraphRAG)," 2025.

[61] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 4171–4186.

[62] F. Yan, P. Yu, and X. Chen, "LTNER: Large language model tagging for named entity recognition with contextualized entity marking," in *International Conference on Pattern Recognition*. Springer, 2024, pp. 399–411.

[63] F. Villena, L. Miranda, and C. Aracena, "llmNER:(Zero\ Few)-Shot Named Entity Recognition, Exploiting the Power of Large Language Models," *arXiv preprint arXiv:2406.04528*, 2024.

[64] T. Xie, Q. Li, Y. Zhang, Z. Liu, and H. Wang, "Self-improving for zero-shot named entity recognition with large language models," *arXiv preprint arXiv:2311.08921*, 2023.

[65] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[66] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, 2023, pp. 31–53.

[67] N. Zhang, Q. Huang, X. Xia, Y. Zou, D. Lo, and Z. Xing, "Chatbot4QR: Interactive query refinement for technical question retrieval," *IEEE Transactions on Software Engineering*, vol. 48, no. 4, pp. 1185–1211, 2020.

[68] A.-R. Preda, C. Mayr-Dorn, A. Mashkoor, and A. Egyed, "Supporting high-level to low-level requirements coverage reviewing with large language models," in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 242–253.

[69] S. Hassani, M. Sabetzadeh, and D. Amyot, "An Empirical Study on LLM-based Classification of Requirements-related Provisions in Food-safety Regulations," *arXiv:2501.14683*, 2025.

[70] Q. Guo, J. Cao, X. Xie, S. Liu, X. Li, B. Chen, and X. Peng, "Exploring the potential of ChatGPT in automated code refinement: An empirical study," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.

[71] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Leveraging GPT-like LLMs to Automate Issue Labeling," in *IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, 2024, pp. 469–480.

[72] L. Shi, X. Chen, Y. Yang, H. Jiang, Z. Jiang, N. Niu, and Q. Wang, "A first look at developers' live chat on Gitter," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 391–403.

[73] R. Tufano, A. Mastropaolo, F. Pepe, O. Dabić, M. Di Penta, and G. Bavota, "Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study," in *IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, 2024, pp. 571–583.

[74] Y. Aquino, P. Bento, A. Buzelin, L. Dayrell, S. Malaquias, C. Santana, V. Estanislau, P. Dutenhefner, G. H. Evangelista, L. G. Porfírio *et al.*, "Discord Unveiled: A Comprehensive Dataset of Public Communication (2015-2024)," *arXiv preprint arXiv:2502.00627*, 2025.